

- 题目
 - 题目描述
 - 输入格式
 - 输出格式
 - 输入输出样例 #1
 - 输入 #1
 - 输出 #1
 - 说明/提示
- 题解
 - 输入代码如下：
 - 对于输入样例，矩阵内部存储情况如下：
 - DFS搜索
 - AC代码

题目

题目描述

在一个地图上有 N ($N \leq 20$) 个地窖，每个地窖中埋有一定数量的地雷。同时，给出地窖之间的连接路径。当地窖及其连接的数据给出之后，某人可以从任一处开始挖地雷，然后每次可以移动到一个编号比当前节点大且联通的节点去挖地雷，当无满足条件的节点时挖地雷工作结束。设计一个挖地雷的方案，使某人能挖到最多的地雷。

输入格式

有若干行。

第 1 行只有一个数字，表示地窖的个数 N 。

第 2 行有 N 个数，分别表示每个地窖中的地雷个数。

第 3 行至第 $N + 1$ 行表示地窖之间的连接情况：

第 3 行有 $n - 1$ 个数 (0 或 1)，表示第一个地窖至第 2 个、第 3 个 ... 第 n 个地窖有否路径连接。如第 3 行为 1 1 0 0 0 ... 0，则表示第 1 个地窖至第 2 个地窖有路径，至第 3 个地窖有路径，至第 4 个地窖、第 5 个 ... 第 n 个地窖没有路径。

第 4 行有 $n-2$ 个数，表示第二个地窖至第 3 个、第 4 个 ... 第 n 个地窖有否路径连接。

.....

第 $n+1$ 行有 1 个数，表示第 $n-1$ 个地窖至第 n 个地窖有否路径连接。(为 0 表示没有路径，为 1 表示有路径)。

输出格式

第一行表示挖得最多地雷时的挖地雷的顺序，各地窖序号间以一个空格分隔，不得有多余的空格。

第二行只有一个数，表示能挖到的最多地雷数。

输入输出样例 #1

输入 #1

```
5
10 8 4 7 6
1 1 1 0
0 0 0
1 1
1
```

输出 #1

```
1 3 4 5
27
```

说明/提示

【样例解释】 最优路径为 $1 \rightarrow 3 \rightarrow 4 \rightarrow 5$ ，结果为 27。

题解

通过读题可以了解到这道题的目的是求有向无环图的权值最大路径。

由于题目所给数据量很小，只有最多20个点，规模非常小，所以可以考虑使用邻接矩阵来存储图，我们开辟一个二维数组 $m[25][25]$ ， $m[i][j]$ 的值如果为1，则代表从第*i*号房间可以去到第*j*号房间。

输入代码如下：

*n*代表共有多少个房间

*w*用来存储每个房间的权值

```
cin>>n; // 房间数
for(int i=1;i<=n;i++) cin>>w[i];           // 每个房间的权值(地雷数)
for(int i=1;i<=n;i++){
    for(int j=i+1;j<=n;j++){
        cin>>m[i][j];                       // 输入值代表第i个房间是否能直接通往第[j]个房间
    }
}
```

对于输入样例，矩阵内部存储情况如下：

	$m[i][1]$	$m[i][2]$	$m[i][3]$	$m[i][4]$	$m[i][5]$
$m[1][j]$	0	1	1	1	0
$m[2][j]$	0	0	0	0	0
$m[3][j]$	0	0	0	1	1
$m[4][j]$	0	0	0	0	1
$m[5][j]$	0	0	0	0	0

DFS搜索

输入完成后，我们可以开始使用dfs搜索来对每个起点寻找其权值最大的路径，通过ans记录最大权值之和，通过path记录权值最大路径，通过cp暂存当前路径

在dfs函数中，我们需要传入两个参数 u和ww，其中u代表当前点，ww代表走到u点时的权值之和。

对于当前点，我们需要访问并搜索它所能走过的每一条路径，通过遍历m[u]可以确定u点与各个点的联通状态，如果联通，我们就进一步地搜索与它联通的下一个点。

如果ww>ans，即当前权值之和大于之前搜索到的答案，那么就需要更新一下之前记录的答案(ans)，同时更新一下之前记录的路径(path)。

```
void dfs(int u,int ww){
    if(ans<ww){                // 如果当前权值之和大于此前记录的值，就更新答案
        ans=ww;
        path=cp;
    }
    for(int v=u+1;v<=n;v++){    // 遍历m[u]确定u与各点的联通状态，若联通则
        进一步搜索
        if(m[u][v]){
            cp.push_back(v);
            dfs(v,ww+w[v]);
            cp.pop_back();
        }
    }
}
```

AC代码

```
#include <bits/stdc++.h>
using namespace std;

int n,ans;        // n 点数,  ans 最大权值
int w[25];        // w[i] i点的权值
int m[25][25];    // m[i][j] i是否可以通往j  若为1则可以, 若为0则不行

vector<int> path,cp;    // path 答案路径  cp 暂存当前路径

void dfs(int u,int ww){
    if(ans<ww){
        ans=ww;
        path=cp;
    }
    for(int v=u+1;v<=n;v++){
        if(m[u][v]){
            cp.push_back(v);
```

```

        dfs(v, ww+w[v]);
        cp.pop_back();
    }
}

void solve(){
    cin>>n;
    for(int i=1;i<=n;i++) cin>>w[i];

    for(int i=1;i<=n;i++)
        for(int j=i+1;j<=n;j++)
            cin>>m[i][j];

    for(int i=1;i<=n;i++) m[0][i]=1;
    dfs(0,0);

    for(auto x : path) cout<<x<<' ';
    cout<<'\n';
    cout<<ans;
}

int main(){
    ios::sync_with_stdio(0);
    cin.tie(0);cout.tie(0);

    int qaq=1;// cin>>qaq;
    while(qaq--) solve();
    return 0;
}

```